

Identify the addressing mode and target address if the instruction is 5012030 where $PC = 2000$, $B = 5030$, $X = 3000$

0000 0001 0010 0000 0011 0000
ni x6pe 0 3 0
relative

$i=1$, so immediate addressing.

$$TA = \text{disp} + (P_c) = 0.30 + 2000 = 2030 //$$

Define assembler, assembler directives and their uses in assembly language program.

i) Assembly:

→ It is a system software that translates an assembly language program into machine code that can be executed by the computer. It also generates information such as the symbol table, location counter and object program.

- Assigns address to instructions and data.

Resolves symbolic addresses.

Generates object program.

Reports syntax and semantic errors.

Assembler directives:

They are instructions given to the assembler to control the assembly process. They don't generate machine code but guide the assembler in allocating memory, defining constants and controlling program organization.

Eg:

→ **START** : specifies the starting address of the program.

→ END: Marks the end of the source program.

→ WORD: Defines a 3 byte integer constant

→ RES W: Reserve a specified number of words

Eg: COPY START 1000 ~ Program starts at 1000
 FIRST LDA ALPHA
 ADD ONE
 STA BETA
 ALPH WORD 5 ~ WORD stores integer constants
 ONE WORD 1
 BETA RESW 1
 END ~~FIRST~~ COPY ~ Indicates end of The program

2.5)

Line	LOCTR	SYMBOL	OPCODE	Exp	OBJ CODE
10	—	TEST	START	0	—
20	0000	FIRST	+LDT	#12	75 ¹ 00012
30	0004		LDX	#0	050000
40	0002	MOVECH	LDCH	STR1,X	57A023 53A008
50	000A		STCH	STR2,X	57A023
60	000D		TIYA	T	B850
70	000F		JLT	MOVECH	3B2FF5
80	0012	STR1	RESW	10	—
90	0030	STR2	BYTE	x'12'	'12
100	0031		END	FIRST	—

OP TAB

Mnemonic	Op code
TIYA	B8
LDT	74
LDCH	50
LDX	04
STCH	54
JLT	38

SYMTAB

SYMBOL	Addr
FIRST	0000
MOVECH	0002
STR1	0012
STR2	0030

20) LDT #12

74

$$0111 \quad 0101 \quad 0001 \quad 012 = 75100012$$

30) LDY #0

04

$$0000 \quad 0101 \quad 0000 \quad 000 = 050000$$
~~Disp~~

40) LDCH STR1, X

50 0012

$$0101 \quad 0011 \quad 1010 \quad 008 = 53A008$$

$$Disp = 0012 - 000A = 008$$

50) STCH STR2, X

54 0030

$$0101 \quad 0111 \quad 1010 \quad 023 = 57A023$$

$$Disp = 0030 - 000D = 023$$

60) TIXR T

B8 5 = B150

70) JLT MOVECH

38 0007

$$0011 \quad 1011 \quad 0010 \quad FF5 = 3B2FF5$$

$$Disp = 0007 - 0012 = FF5$$

80)

Object file:

H ^ Test -- ^ 000000 ^ 000031

T ^ 000000 ^ 13 ^ 75100012 ^ 050000 ^ 53A008 ^ 53A023 ^

B150 ^ 3B2FF5, 12

E ^ 000000

3.a) Explain the machine dependent features with example
= Machine dependent features are those features that depend on the architecture and instruction set of the target machine. The assembler must understand these features to generate the correct object code.

i) Instruction format:

→ Different machines support different instruction formats (instruction length). The assembler determines the correct format & generates the corresponding object code.

Format 1:

opcode

8 bits

Format 2:

opcode	R ₁	R ₂
--------	----------------	----------------

8 bits 4 bits 4 bits

Format 3:

opcode	n	i	x	b	p	e	disp
--------	---	---	---	---	---	---	------

6-bits 1 1 1 1 1 12

Format 4:

opcode	n	i	x	b	p	e	add
--------	---	---	---	---	---	---	-----

6 1 1 1 1 1 20

Eg:

~~LDA~~ ALPH FIX : Format 1

~~LDA~~ TIXR T : Format 2

LDA ALPHA : Format 3

+ LDA ALPHA : Format 4

ii) Addressing modes:

→ Addressing modes specify how the operand is obtained. The assembler sets the n, i, x, b, p, e bits accordingly.

a) Immediate addressing: (n=0, i=1)

Eg: LDA #5

Operand is the constant 5.

b) ~~Addressing mode~~: Indirect addressing: (n=1, i=0)

Eg: LDA @VALUE

Operand is found through the address stored at VALUE.

c) Direct addressing: ($x=0$)

→ Eg: LDA VALUE

Loads the contents stored at VALUE

d) Indexed addressing: ($x=1$)

Eg: LDA TABLE, X

Target address $\Rightarrow$ TA = TABLE + X

e) PC relative addressing: ($P=1$)

Eg: JLT LOOP

TA = PC + displacement

f) Base relative addressing: ($b=1$)

Eg: BASE TABLE

LDA VALUE

TA = B + displacement

iii) Program relocation:

→ A program may need to be loaded at a memory location different from the one assumed during assembly. The assembler provides relocation information so that the loader can adjust address fields. That information is provided using M record

Eg: ~~0000~~ 0002 + JSUB RDAEC
Then, MA 000003, 05

3.b) Why is forward reference a problem for assembler? How does assembler try to solve the forward reference issues?

= A forward reference occurs when a symbol is used before it is defined in the assembly language program.

When the assembler encounters a forward reference, it does not yet know the address of the symbol because it has not been defined.

For eg: LDA VALUE

...
VALUE WORD 10

→ When assembling LDA VALUE, the assembler needs the address of

VALUE to generate the object code. However since VALUE is defined later, it's address is unknown at that moment. This creates problems such as:

- The assembler cannot generate the correct obj code immediately
- The symbol is missing from the SYMTAB.
- Address calculation becomes impossible until the symbol is defined

Address Assembler solves these issues by:

i) Two pass assembler:

→ It solves forward references by scanning the source program twice

Pass 1:

- Assigns address to all statements using the LOCTR.
- Builds the SYMTAB by recording the addresses of all labels.

Pass 2:

- Reads program again & uses the completed SYMTAB to replace symbols with their addresses.
- Generate the correct object code.

ii) Backpatching (Single pass assembler):

→ A single pass assembler cannot reread the program, so it uses block backpatching.

Working:

- When an undefined symbol is encountered, the assembler enters the symbol into SYMTAB with an undefined value; stores the locations where the symbol is referenced in a forward reference list
- When the symbol is eventually defined, the assembler updates its address in SYMTAB.

It goes back and patches (fills in) the previously generated object code at all recorded locations.

4)

Line	LOCTR	SYMBOL	OPCODE	Exp	OBJ CODE
10	—	STRCPY	START	4000	—
20	4000	FIRST	LDX	ZERO	04401E
30	4003	MOVECH	LDCH	STR1,X	50C00F
40	4006		STCH	STR2,X	54C013
50	4009		TSX	ELEVEN	2C4021
60	400C		JLT	MOVECH	384003
70	400F	STR1	BYTE	C'ABCD'	41424344
80	4013	STR2	RESB	11	—
90	4018E	ZERO	WORD	0	000000
100	40201	ELEVEN	WORD	11	00000B
110	40234		END	FIRST	—

H A STRCPY A 004000 A 000024

T A 004000 A 19 A 04401E A 50C00F A 54C013 A 2C4021 A 384003 A
41424344 A 000000 A 00000B

E A 004000

00010203 04050607 08 09 A B C D E F
XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX

0000

04000 04401E50 C00F54C0 132C4021 38400341

04010 424344xx xxxxxxxx xxxxxxxx yxxx0000

4020 0000000B xxxxxxxx xxxxxxxx xxxxxxxx

5.a) How does conditional macro expansion handled? Explain mechanism of concatenation of macro parameter.

= Conditional macro expansion is a feature of macro processor that allows different sets of statements to be expanded based on a specified condⁿ. It works like an IF ELSE statement in high level language.

- The macro processor evaluates the condⁿ during macro expansion
- A macro is called with actual parameters.
 - The macro processor substitutes the parameters into the macro.
 - It evaluates the conditional expression
 - If condⁿ is true, it expands the corresponding statements.
 - If condⁿ is false, it expands the alternative statements (if any)
 - Expansion continues until the end of macro.

Example:

```
MACRO LOAD &TYPE, &VAR
    IF (&TYPE EQ CHAR)
        LDCH &VAR
    ELSE
        LDA &VAR
    MEND ENDC
MEND
```

Macro call: LOAD CHAR, STR1

→ LDCH STR1

5.b) Explain the working principle of concatenation of macro parameter and generation of unique labels in the machine independent macro processor features.

- = Concatenation of macro parameters is the process of joining a macro parameter with other characters or symbols to form a new identifier during macro expansion.

Working principle:

- i) The macro processor reads the macro definition
- ii) During macro expansion, it substitutes the actual parameter for the formal parameter
- iii) The parameter is concatenated with fixed text to create a new symbol or variable name.
- iv) The expanded code contains the newly formed identifier

Eg: MACRO STORE &NUM
STA VALUE&NUM
MEND

macro call: STORE 1
→ STA VALUE1

Generation of unique labels:

- When a macro containing labels is expanded multiple times, duplicate labels may occur. To avoid this, the macro processor automatically generates unique labels for each macro expansion.
- The macro processor replaces each label with a unique generated label after using symbols like &AA, &AB etc.

Eg:

MACRO LOOPM
LOOP ~~LDA~~ LDA VALUE
TIX COUNT
JLT LOOP
MEND

Expansion 1:

\$AALOOP LDA VALUE
TIX COUNT
JLT \$AALOOP

Expansion 2:

\$ABLOOP LDA VALUE
TIX COUNT
JLT \$ABLOOP

Advantages:

- Prevents duplicate label errors.
- Allows the same macro to be expanded multiple times.
- Ensures correct branching within each macro expansion.

6-a) What do you ^{understand} mean by the grammar? How do you generate the strings from a grammar?

= The Grammar is a set of rules that defines how a program must be written in a programming language. It specifies the order of identifiers, variables, constants, operators, values and keywords.

A grammar can be written in various forms such as BNF, CFG.

→ Grammar consists of Non terminals, terminals, production rule, starting point. i.e. $G = (N, T, P, S)$

i) Non terminals:

- They are variables that describe the structure of a language.
 - It can be further expanded or divided.
 - They don't appear in the program.
- Eg: statements, sub routine, expressions etc.

ii) Terminals:

- They can't be further expanded or divided.
 - They are actual symbols or tokens that appear in a program.
- Eg: Identifier, +, *, while, for etc.

iii) Starting point:

- It is the first non terminal from where parsing begins.

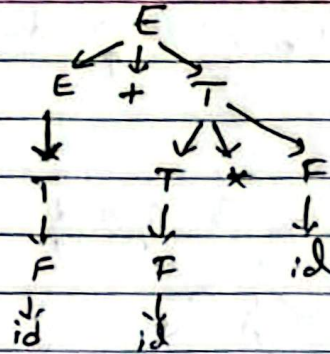
iv) Production rule:

- It describes how one symbol can be replaced by another.
- Eg: left → right which means left can be replaced by right.
- Strings are generated by repeatedly applying the production rules, starting from the start symbol until only terminal symbols remain.

Eg: $E \rightarrow E + T \mid T$

$T \rightarrow F \mid T * F$

$F \rightarrow id$



7a) Recursive descent parsing.

- It is a top down parsing technique in which a set of recursive procedures is used to parse the input string.
- Each non terminal in the grammar is represented by a separate recursive functions. The parser starts from the start symbol and recursively expands productions until the input string is successfully matched.

Working:

- Start parsing from the start symbol.
- Call the recursive function corresponding to the current non terminal.
- Match terminals symbol with the input string.
- If mismatch occurs, report a syntax error (or backtrack).
- Parsing is successful when the entire input is consumed.

Advantages

- Easy to implement
- Simple
- Easy to debug

Disadvantages

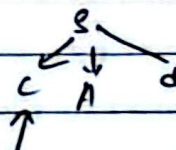
- Slower
- Less powerful
- May require grammar modification

Eg: $S \rightarrow CAD$

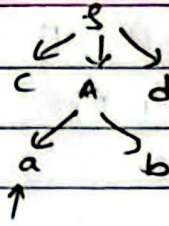
$A \rightarrow ab \mid a$

Validate cad,

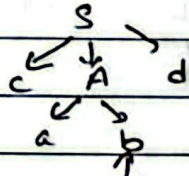
Step 1: cad



Step 2: c a d
 ↑

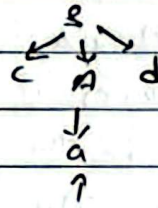


Step 3: c a d
 ↑

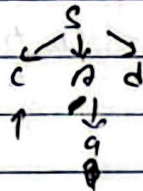


Now backtrack,

Step 4: c a d
 ↑



Step 5: c a d
 ↑



7. b) Relocation bit mask:

- A relocation bit mask is a sequence of bits included in the object program that indicates which instructions or address fields need to be modified when a program is loaded at a memory location different from the one assumed during assembly working:

The assembler generates one relocation bit for each word in a text record.

Bit = 1 : The address is relocatable and must be modified by loader

Bit = 2 : The corresponding value is absolute & does not require modification.

Example:

OPCODE	OPERAND	Bit
WORD LDA	ALPHA	1
ADD	ONE	1
LDA	BETA	1
ALPHA	WORD 5	5 0
ONE	WORD 2	2 0

The relocation bit mask is 11100

1110 0000 0000

E 0 0 = E00

Advantages:

- Reduces the number of modification records
- Makes relocation faster & more efficient
- Saves space in obj program

Disadvantages:

- Applicable only when instructions have a fixed length (150)
- Not suitable for architectures with ~~some~~ variable length instructions such as PIC/XE where modification records (M) are used instead